BITS Pilani
DIGITAL
Excellence made yours

Scan QR for Project Brochure

# Bill Scanning Based Reward & Intelligence System

Final Study Project Report

Phase “3”

Presented By **Team ARAJ** – Group (120)
Advised By **Prof. Uma Sankara Rao**

# Project Details:

| | | |
|---|---|---|
| Course Title | B.Sc. Computer Science – BITS Pilani Digital | |
| Project Title | Bill Scanning Based Reward & Intelligence System | |
| Phase | 3 | |
| Group No. | 120 | |
| Gropu Name | Team ARAJ | |
| Number of members | 4 | |
| Members & ID | Ashfaaq Feroz Muhammad | 2023EBCS005 |
| | Ranjeet Singh | 2023EBCS457 |
| | Arpan Chatterjee | 2023EBCS399 |
| | Jyoti Kataria | 2023EBCS435 |
| Group Advisor | Prof. Uma Sankara Rao | |
| Submission Date | 03 \| March \| 2026 | |

# Team ARAJ

Who We Are

Team ARAJ, Group 120, is a four-member student team from the B.Sc. Computer Science program (BITS Pilani Digital). This study project is completed as part of our final-year academic requirements, emphasizing real-world problem identification and the creation of feasible, technology-driven solutions.

Our chosen project area is Artificial Intelligence and Machine Learning (AI/ML), focusing on data extraction, pattern recognition, and converting unstructured data into meaningful, privacy-compliant intelligence.

**Roles:**

- → **Idea:** Arpan Chatterjee
- → **Idea Executing Planner:** Jyoti Kataria
- → **Lead Developer:** Ashfaaq Feroz Muhammad
- → **Assisted Developer:** Ranjeet Singh

# Bill Scanning–Based Reward & Intelligence System

Team ARAJ — **A**shfaaq, **R**anjeet, **A**rpan, **J**yoti

## Phase 3: Implementation Readiness & Validation

### 1. Introduction

### 1.1 Purpose of Phase 3

The primary objective of Phase 3 is to:

- Demonstrate the implementation readiness and architectural stability of the "Bill Scanning Based Reward & Intelligence System."
- focuses on validating critical technical design choices by successfully integrating Optical Character Recognition (OCR), Generative Artificial Intelligence (Gemini 2.5 Flash).
- cloud-based data persistence (Firebase Firestore) via a custom Node.js Express server backend.
- It aims to establish system reliability through robust error handling, assess current API constraints, and solidify the structural foundation necessary for future scalable deployment.

**Summary**

Phase 3 validates architectural stability by integrating OCR and Gemini AI with Firebase. It focuses on implementation readiness, robust error handling, and solidifying a reliable foundation for future scalable deployment.

## 1.2 Summary of Work Completed So Far

- **Phase 1 Outcomes:** Defined the core problem associated with manual receipt processing and mapped the initial technical ecosystem requirements, feasibility, and project scope.

- **Phase 2 Outcomes:** Established the proof-of-concept for a deeply engaging, unique user interface (UI). Designed the gamified reward hierarchy (incorporating interactive scratch cards and voucher mechanisms) and architected the initial backend database schema.

# 2. Implementation Overvie

## 2.1 Implementation Status:

- **Fully Implemented Modules:** A highly optimized and unique frontend user interface, Express.js Backend Routing, Gemini 2.5 AI Data Extraction Pipeline, Live Firebase Firestore Integration (CRUD Operations), and Gamified Reward Logic execution engines.
- **Partially Implemented Modules:** User Authentication is currently functioning securely via local JSON Web Tokens (JWT) mapped to single-user Firestore architectures for the purpose of the Proof of Concept (PoC).
- **Planned but Not Implemented Components:** Direct third-party merchant API integrations (Simulated organically within the current application state to maintain project scope).

## 2.2 Implemented Features

- **AI Receipt Parsing:** Real-time multimodal extraction of Merchant Names, Dates, Total Amounts, and Item Categories matching strict JSON schemas using Google's Gemini SDK.
- **Database Persistence:** Secure HTTP connections to Firebase Firestore mapping interconnected collections for Users, Merchants, Receipts, Claimed Rewards, and Consent Logs.
- **Advanced Error Handling Algorithms:** Built-in validation algorithms to gracefully return HTTP 422 Unprocessable Entity (blurry image detection), HTTP 409 Conflict (duplicate receipt detection), and HTTP 429 Too Many Requests (handling API rate limits safely without crashing the backend).
- **Tiered Reward Engine:** Dynamic point multiplication algorithms calculating reward yields based on user streaks and vendor categories.
- **Analytics Dashboard:** Intelligent mock insights derived algorithmically from historic user spending patterns and category distributions.

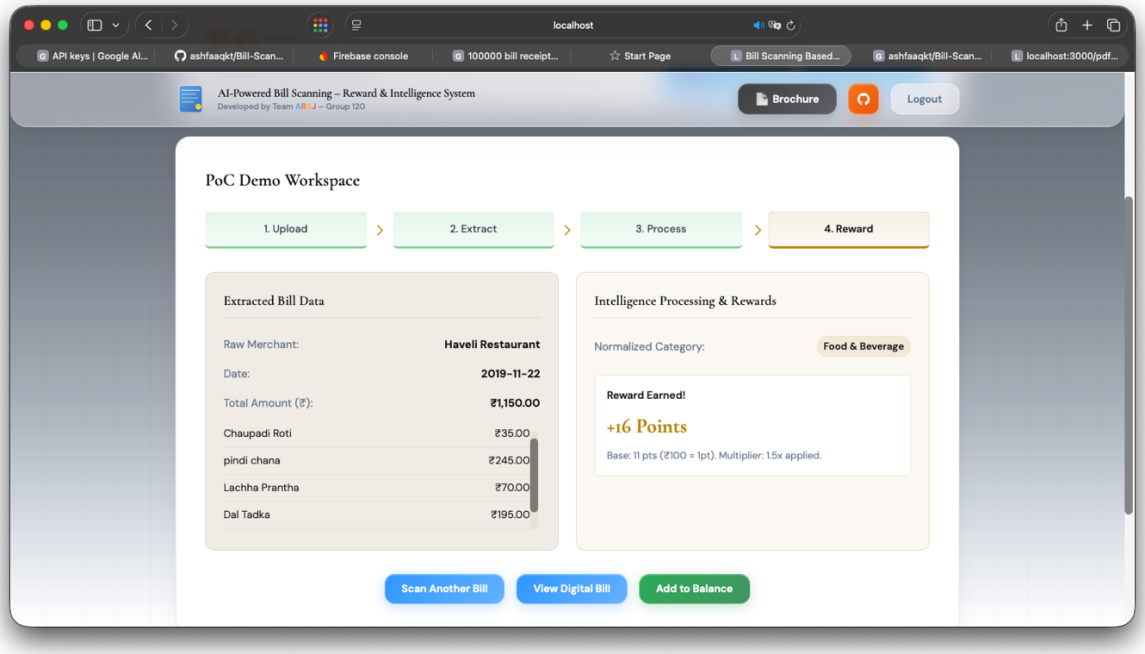
# 3. System Validation and Testing:

## 3.1 Testing Strategy

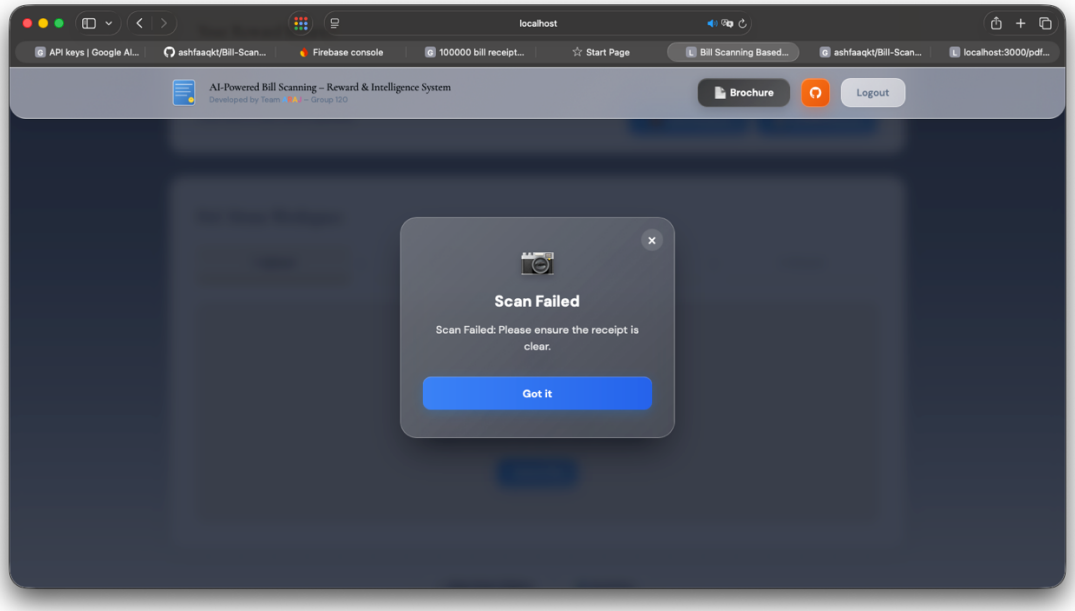
The system underwent rigorous component and integration testing methodologies. The testing strategy focused extensively on the Express.js backend-to-frontend communication pipeline, specifically targeting the `/api/upload` endpoint to validate Gemini JSON parsing accuracy and latency under various network states.

## 3.2 Test Cases and Results

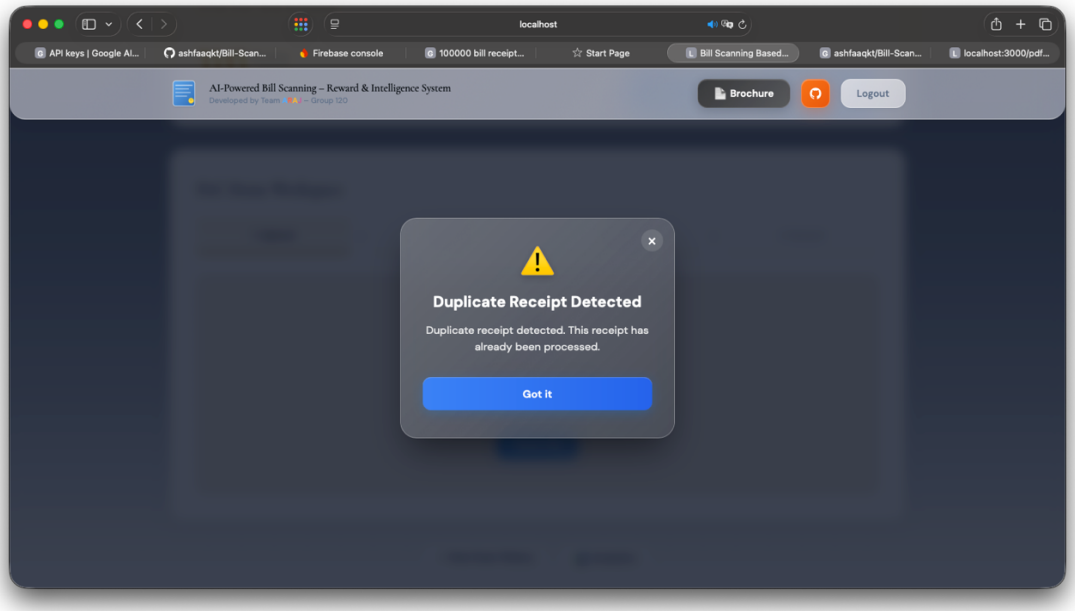
- **Test Case 1:** Upload Valid Resturent Receipt.
  - → Expected Behavior: HTTP 200 OK + Extracted JSON Data.
  - → Observed Result: Pass. System successfully extracted and strictly typed the data.



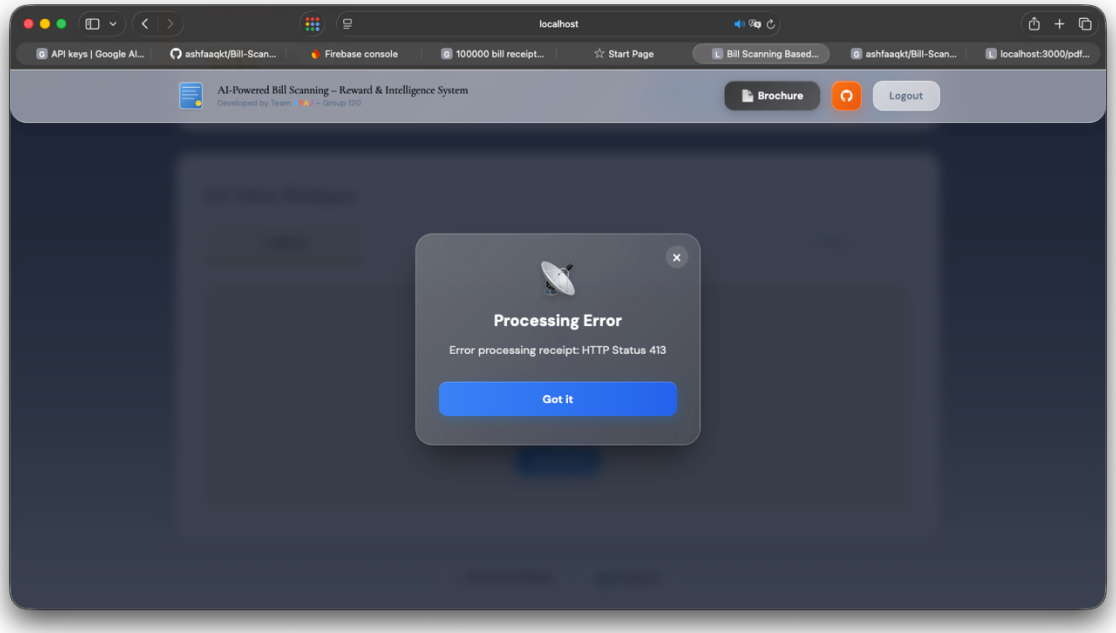
- **<u>Test Case 2:</u>** Upload Blurry/Illegible Receipt Image.
  - → Expected Behavior: HTTP 422 Unprocessable Entity via AI Rejection.
  - → Observed Result: Pass. Custom error modal accurately prompted the user.




- **<u>Test Case 3:</u>** Upload Exact Duplicate Receipt.
  - → Expected Behavior: HTTP 409 Conflict via Firestore indexing query.
  - → Observed Result: Pass. The system identified identical Merchant, Date, and Total values.



- Test Case 4: Google API Quota Exhaustion (Rate Limiting).
  - → Expected Behavior: HTTP 429 Rate Limit Warning intercept.
  - → Observed Result: Pass. The system successfully caught the internal API rejection and propagated a graceful UI state to the user without experiencing a generic server crash.




## 3.3 Validation Summary

The system successfully meets all defined Phase 3 operational requirements. The most significant technical achievement during this phase was the aggressive programmatic mitigation of non-deterministic AI hallucinations. The backend now utilizes defensive substring extraction programming to accurately isolate valid JSON payloads from surrounding conversational AI outputs, ensuring a highly durable database parse success rate on the `/api/upload` route.

## 4. Performance and Reliability Analysis

- **Responsiveness:** The unique UI interface is heavily optimized using modern CSS rendering techniques, ensuring it runs efficiently and responsively across various desktop and mobile viewports.
- **Stability:** The Node.js asynchronous architecture handles parallel API connections efficiently, decoupling frontend UI thread blocking from heavily latent AI processing tasks.
- **Resource Bottlenecks:** The primary constraint observed is the Free-tier Google Gemini 2.5 Flash API frequency limit (e.g., 15 Requests Per Minute), which artificially throttles overall system throughput during peak parallel validations.

## 5. Risk Analysis and Mitigation Review

### 5.1 Identified Risks (Revisited)

- The technical complexity of safely parsing non-deterministic and occasionally unpredictable text sequences returned by Generative Language Models.
- Over-reliance on third-party cloud architectures (Firebase & Google AI infrastructure).

### 5.2 Mitigation Effectiveness

- The paramount risk of AI hallucination corrupting the backend was successfully mitigated by deploying aggressive bounds-checking sequences (searching for strict JSON object encapsulations) prior to attempting standard JSON parsing.
- Third-party dependency risks remain pronounced. Free-tier quota limits inherently force a hard ceiling on systematic load testing until a transition is made to premium cloud tiers.

## 6. Limitations and Constraints

- **Server Dependency Limits:** Static hosting environments (such as GitHub Pages) strictly host frontend assets. They lack the continuous runtime environment required for Experss.js, meaning local machine execution is required to launch the complete API suite.
- **Generalist Model Constraints:** Because Gemini is a generalized multimodal LLM, it occasionally requires exhaustive prompt-engineering to correctly identify highly unique or regionally specific Indian retail billing formats.
- **Network Latency:** The necessity of transmitting base64 encoded image strings to Google's cloud introduces inherent latency spikes dependent on the user's local network upload bandwidth.

## 7. Future Enhancements and Scope Extension

→ **Independent Cloud Deployment:** Containerizing the backend and transitioning the Node.js architecture to a dedicated VPS (e.g., Render or AWS EC2 instances) to decouple the APIs from local dependencies.

→ **Targeted Model Customization:** Investigating the feasibility of fine-tuning a localized, lightweight LLM specifically trained on diverse Indian retail billing formats to increase accuracy speeds while reducing dependency on global APIs.

→ **Vendor Integrations:** Building secure B2B partner portals allowing Demand-Side Platforms (DSPs) to purchase anonymized user analytics legally.

→ **API & SDK Development:** Creating robust APIs and dedicated SDKs to allow seamless integration of our intelligent reward and OCR systems into third-party retail applications.

→ **Brand Collaboration & Data Sharing:** Partnering and collaborating with brands to share aggregated, anonymized consumer spending data, enabling them to offer highly targeted rewards and collaborative campaigns.

## 8. Learning Outcomes and Reflections

**Over the course of Phase 3, the team mastered:**

1. Complex asynchronous JavaScript pipelines, seamless API error propagation,
2. Modern CSS architecture.
3. The most profound learning outcome involved navigating the erratic nature of Generative AI integration in a production environment.
4. Prompt engineering alone proved insufficient, confirming the critical necessity of deploying defensive backend programming methodologies to force third-party AI interfaces into predictable stability.

## 9. Final Deliverables

→ **Complete Source Code Repository (Version Controlled)** [Visit GitHub]

→ **Full Stack Executable Files (Runnable via Node.js local environments)**

→ **The interactive, unique user interface demonstrating the PoC workflow**

## 10. Conclusion

The Phase 3 Implementation Readiness goals have been successfully fulfilled. This Proof of Concept operates seamlessly, bridging a highly unique modern frontend integration with advanced Generative AI backend pipelines and secure cloud data tracking. Team ARAJ has successfully built a stable, resilient, and highly scalable foundation that is fully prepared for comprehensive academic evaluation and serves as a blueprint for iterative production deployments.

**Supplementary Materials and Local Validation:**
Detailed visual documentation, including comprehensive screen recordings and interface screenshots, is accessible within the project's GitHub repository. Academic evaluators and technical reviewers are encouraged to clone the repository to independently execute the system, validate the documented test cases, and interact with the application firsthand.

**The End**
**Thank You..**

Advisor Feedback:

Supervisor Comments:

Recommendations:

| Submission Date | Group Advisor | TEAM |
|---|---|---|
| 03-Mar-2026 | Prof. Uma Sankara Rao | ARAJ |